

Lezione: Introduzione a Node.js

Cos'è Node.js?

- Ambiente di esecuzione JavaScript lato server
- Basato sul motore V8 di Chrome
- Permette di sviluppare applicazioni di rete scalabili e performanti

Caratteristiche principali

- Event-driven, non-blocking I/O
- Single-threaded con gestione asincrona
- Ampio ecosistema di moduli tramite npm

Installazione di Node.js

1. Scarica l'installer dal sito ufficiale: nodejs.org
2. Segui le istruzioni per il tuo sistema operativo (Windows, macOS, Linux)
3. Verifica l'installazione:

```
node -v  
npm -v
```

Primo esempio: Hello World

```
// hello.js  
console.log("Hello, Node.js!");
```

Esegui con:

```
node hello.js
```

Gestione dei pacchetti con npm

- `npm init` per creare un nuovo progetto
- `npm install <package>` per aggiungere dipendenze
- `package.json` per gestire le informazioni del progetto

Esercitazione: Creare un semplice server HTTP

```
const http = require("http");
const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Ciao dal server Node.js!");
});
server.listen(3000, () => {
  console.log("Server in ascolto su http://localhost:3000");
});
```

Moduli core di Node.js

Node.js offre diversi moduli integrati per funzionalità comuni:

- `fs` : gestione del filesystem
- `path` : gestione dei percorsi dei file
- `http` : creazione di server e client HTTP
- `os` : informazioni sul sistema operativo

Esempio: Lettura di un file con `fs`

```
const fs = require("fs");

fs.readFile("file.txt", "utf8", (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

Esempio: Uso di `path`

```
const path = require("path");  
  
const filePath = path.join(__dirname, "file.txt");  
console.log(filePath);
```

Esempio: Informazioni sul sistema con `os`

```
const os = require("os");  
  
console.log("Platform:", os.platform());  
console.log("CPU:", os.cpus().length);
```

Gestione dei pacchetti e script con npm

- `npm install` installa tutte le dipendenze elencate in `package.json`
- Puoi aggiungere script personalizzati nella sezione `"scripts"` di `package.json`

Esempio di `package.json` minimale:

```
{
  "name": "esempio-node",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  }
}
```

Per eseguire uno script:

```
npm run start
```

oppure, per script predefiniti come `start` :

```
npm start
```

- Usa `npm install nodemon --save-dev` per installare strumenti di sviluppo come `nodemon` (riavvio automatico del server).
- Le dipendenze vengono gestite automaticamente e salvate in `node_modules/` e `package-lock.json`.

NPM

- **npm** (Node Package Manager) è il gestore di pacchetti ufficiale di Node.js.
- Permette di installare, aggiornare e gestire librerie e strumenti per i tuoi progetti.
- Il sito ufficiale per cercare e scaricare pacchetti è <https://www.npmjs.com/>.

Come cercare pacchetti

- Vai su [npmjs.com](https://www.npmjs.com/) e usa la barra di ricerca per trovare librerie.
- Ogni pacchetto ha una pagina dedicata con documentazione, comandi di installazione e dettagli sulle versioni.

Installazione di un pacchetto

- I pacchetti vengono aggiunti automaticamente a `package.json` e scaricati nella cartella `node_modules/`.

Esempio: installare Express

```
npm install express
```

- Dopo l'installazione, puoi importare il pacchetto nel tuo codice:

```
const express = require("express");
```

Callback, Promises e async/await

Node.js gestisce le operazioni asincrone con diversi approcci:

- **Callback:** funzione chiamata al termine di un'operazione
- **Promise:** oggetto che rappresenta il risultato futuro di un'operazione
- **async/await:** sintassi moderna per lavorare con le Promise

Esempio: Callback

```
fs.readFile("file.txt", "utf8", (err, data) => {  
  if (err) throw err;  
  console.log(data);  
});
```

Esempio: Promise

```
const fs = require("fs").promises;  
  
fs.readFile("file.txt", "utf8")  
  .then((data) => console.log(data))  
  .catch((err) => console.error(err));
```

Esempio: async/await

```
const fs = require("fs").promises;

async function leggiFile() {
  try {
    const data = await fs.readFile("file.txt", "utf8");
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}

leggiFile();
```

Risorse utili

- [Documentazione ufficiale Node.js](#)
- [NodeSchool](#)
- [npm documentation](#)

Lezione: Introduzione a React

Cos'è React?

- Libreria JavaScript per costruire interfacce utente (UI)
- Sviluppata da Facebook
- Basata su componenti riutilizzabili
- Aggiornamento efficiente del DOM tramite Virtual DOM

Perché usare React?

- Sviluppo UI modulare e scalabile
- Aggiornamenti efficienti e reattivi
- Grande ecosistema e community
- Facilità di integrazione con altre librerie

Creare una nuova app React

1. Installa Node.js e npm

2. Usa Create React App:

```
npx create-react-app mia-app  
cd mia-app  
npm start
```


Struttura di base di un progetto React

- `/src` : codice sorgente dell'app
- `App.js` : componente principale
- `index.js` : punto di ingresso dell'app

Il primo componente React

```
function Welcome() {  
  return <h1>Ciao, React!</h1>;  
}
```

- Un componente è una funzione che restituisce JSX

Cos'è JSX?

- Estensione della sintassi JavaScript
- Permette di scrivere HTML dentro JavaScript

```
const elemento = <h1>Hello, world!</h1>;
```

Rendering di un componente

```
import React from "react";  
import ReactDOM from "react-dom/client";  
import App from "../App";  
  
const root = ReactDOM.createRoot(document.getElementById("root"));  
root.render(<App />);
```

Creare un nuovo componente React: passo passo

1. Crea un nuovo file

Crea un file chiamato `Saluto.js` nella cartella `src`.

2. Scrivi il componente

```
// src/Saluto.js
function Saluto() {
  return <h2>Ciao, sono un nuovo componente!</h2>;
}

export default Saluto;
```

3. Importa e usa il componente in `App.js`

```
import Saluto from "../Saluto";

function App() {
  return (
    <div>
      <h1>Benvenuto in React</h1>
      <Saluto />
    </div>
  );
}

export default App;
```

4. Avvia l'app

Esegui:

```
npm start
```

Vedrai il messaggio del nuovo componente nella pagina.

Props: passare dati ai componenti

```
function Saluto(props) {  
  return <h1>Ciao, {props.nome}!</h1>;  
}
```

// Uso:

```
<Saluto nome="Rocco" />
```


Challenge: Props

Obiettivo: Crea un componente `Messaggio` che accetta una prop `testo` e la visualizza in un paragrafo.

```
function Messaggio({ testo }) {  
  return <p>{testo}</p>;  
}
```

// Uso:

```
<Messaggio testo="Questo è un messaggio passato come prop!" />
```

Esercizio: Creare una pagina HTML con React

Obiettivo: Crea una pagina React che abbia un componente `Header`, un componente `Main` e un componente `Footer`. Ogni componente deve visualizzare un testo diverso.

Esempio di struttura:

```
function Header() {  
  return <header><h1>Benvenuto nella mia pagina!</h1></header>;  
}  
  
function Main() {  
  return <main><p>Questo è il contenuto principale della pagina.</p></main>;  
}  
  
function Footer() {  
  return <footer><small>© 2024 Il tuo nome</small></footer>;  
}
```

```
function App() {  
  return (  
    <div>  
      <Header />  
      <Main />  
      <Footer />  
    </div>  
  );  
}  
  
export default App;
```

- Prova a personalizzare i testi e lo stile dei componenti.
- Puoi aggiungere i componenti in file separati per una migliore organizzazione.
- Avvia l'app per vedere la pagina completa

State: stato locale del componente

```
import { useState } from "react";

function Contatore() {
  const [conteggio, setConteggio] = useState(0);

  return (
    <div>
      <p>Conteggio: {conteggio}</p>
      <button onClick={() => setConteggio(conteggio + 1)}>Aumenta</button>
    </div>
  );
}
```

Challenge: State

Obiettivo: Crea un componente `Toggle` con un bottone che mostra/nasconde un testo.

Gestione degli eventi

```
function ClickMe() {  
  function handleClick() {  
    alert("Hai cliccato!");  
  }  
  
  return <button onClick={handleClick}>Cliccami</button>;  
}
```

Esercizio: Gestione eventi

Obiettivo: Crea un componente con un input e un bottone. Al click, mostra un alert con il testo inserito.

Liste e rendering dinamico

```
const nomi = ["Anna", "Luca", "Marco"];

function ListaNomi() {
  return (
    <ul>
      {nomi.map((nome, idx) => (
        <li key={idx}>{nome}</li>
      ))}
    </ul>
  );
}
```


Challenge: Liste

Obiettivo: Crea un componente che riceve una lista di numeri come prop e visualizza solo quelli pari.

useEffect

/**

- Questo componente React utilizza l'hook useEffect per eseguire effetti collaterali dopo il rendering.
- L'hook useEffect viene spesso utilizzato per gestire operazioni asincrone, come il recupero di dati da un'API,
- la sottoscrizione a eventi, o la manipolazione diretta del DOM. In questo caso, useEffect garantisce che il codice all'interno venga eseguito dopo che il componente è stato montato o aggiornato, in base alle dipendenze specificate.

*/

Uso di useEffect

```
import { useEffect, useState } from "react";

function Timer() {
  const [secondi, setSecondi] = useState(0);

  useEffect(() => {
    const id = setInterval(() => setSecondi(s => s + 1), 1000);
    return () => clearInterval(id);
  }, []);

  return <p>Secondi: {secondi}</p>;
}
```

Esercizio: useEffect

Obiettivo: Crea un componente che mostra la data e ora attuale e si aggiorna ogni secondo.

Componenti figli e composizione

```
function Card({ titolo, children }) {  
  return (  
    <div className="card">  
      <h2>{titolo}</h2>  
      {children}  
    </div>  
  );  
}  
  
// Uso:  
<Card titolo="Benvenuto">  
  <p>Questo è il contenuto della card.</p>  
</Card>
```

Challenge: Composizione

Obiettivo: Crea un componente `Layout` che accetta header, main e footer come children.

Condizionali in React

```
function Messaggio({ isLoggedIn }) {  
  return (  
    <div>  
      {isLoggedIn ? <p>Benvenuto!</p> : <p>Effettua il login.</p>}  
    </div>  
  );  
}
```

Esercizio: Condizionali

Obiettivo: Crea un componente che mostra "Buongiorno" se l'ora è prima delle 12, altrimenti "Buonasera".

Styling in React

- Inline styles:

```
const stile = { color: "blue", fontWeight: "bold" };  
<p style={stile}>Testo colorato</p>
```

- CSS modules o file CSS tradizionali

Challenge: Styling

Obiettivo: Applica uno stile personalizzato a un bottone in base allo stato (attivo/disattivo).

Importare librerie esterne

```
npm install axios
```

```
import axios from "axios";
```

Esercizio: Chiamata API

Obiettivo: Crea un componente che recupera e mostra dati da un'API pubblica (es. JSONPlaceholder).

React DevTools

- Estensione per Chrome/Firefox per ispezionare componenti React
- Utile per debug e ottimizzazione

Risorse utili

- [Documentazione ufficiale React](#)
- [React Tutorial](#)
- [Awesome React](#)